DATABASE MANAGEMENT SYSTEM

ASSIGNMENT 3

DATABASE DESIGN ANALYSIS

Hotel Management System

Particular Details	**Information**
Student Name	A Praisy Emberline
Programme	BCA – Artificial Intelligence & Machine Learning
Subject	Database Management System
University	Alliance University
Academic Year	2nd Year

Introduction

Database design analysis helps identify data redundancy, insertion anomalies, deletion anomalies, and update anomalies. It also examines functional dependencies and suggests improvements to make a database accurate and easy to maintain. In this assignment, the Hotel Management System from the reference is analysed.

1. Data Redundancy and Potential Anomalies

Data Redundancy

- Data redundancy occurs when the same information is stored repeatedly in a database.
- If guest names, room details, or room-type prices are repeated in every reservation row, the database uses extra storage and can become inconsistent.
- The Hotel Management System reduces redundancy by keeping guest details in Guest, room categories in Room_Type, individual rooms in Room, and booking details in Reservation.

Insertion Anomaly

An insertion anomaly occurs when information cannot be added without entering unrelated information.

Example: If room details and guest reservations were stored in one table, a new room might not be recorded until a guest reserves it.

Improvement: Store room information separately in Room and booking information in Reservation.

Deletion Anomaly

A deletion anomaly occurs when deleting one record unintentionally removes other useful information.

Example: If a service name and its price were stored only in Service_Usage, deleting the last usage record could also remove the only stored details of that service.

Improvement: Keep service definitions in Service and individual service-use records in Service_Usage.

Update Anomaly

An update anomaly occurs when the same fact is stored in several rows and every copy must be changed.

Example: If a room type's nightly price is repeated in every reservation, a price change may require updating many records.

Improvement: Store the standard price once in Room_Type and connect rooms using room_type_id.

2. Functional Dependencies Between Attributes

A functional dependency means that one attribute determines another attribute. It is written as X → Y. In this design, each primary key determines the remaining attributes in its table.

Table	Functional Dependency
Guest	guest_id → guest_name, email, phone, address
Room_Type	room_type_id → type_name, price_per_night, description
Room	room_id → room_number, floor_number, room_status, room_type_id
Staff	staff_id → staff_name, role, phone, department
Reservation	reservation_id → guest_id, room_id, check_in_date, check_out_date, reservation_status
Service	service_id → service_name, service_price, description
Payment	payment_id → reservation_id, payment_date, amount, payment_method, payment_status
Service_Usage	usage_id → reservation_id, service_id, usage_date, quantity

3. How Poor Database Design Can Lead to Anomalies

- Insertion anomaly: a new room type may be difficult to add if room-type details are stored only with actual rooms.
- Deletion anomaly: deleting a reservation could accidentally delete guest or room information if everything is stored in one combined table.

- Update anomaly: a guest's phone number could differ between reservations if it is copied into every booking record.

4. Proposed Improvements to the Database Design

- Separate guest, room, room type, staff, reservation, service, payment, and service-usage data into their own tables.
- Use primary keys to identify each row uniquely and foreign keys to connect related records.
- Use UNIQUE constraints for fields that should not be duplicated, such as guest email, if that is a business rule.
- Use CHECK constraints for positive prices and quantities and for check-out dates that occur after check-in dates.
- Keep room-type pricing in Room_Type rather than repeating it in every reservation.
- Store services separately and record each use through Service_Usage, which acts as an associative table.

Conclusion

The Hotel Management System design reduces repeated information by separating data into eight related tables. Functional dependencies show how primary keys identify each record and determine its other attributes. Separating entities, applying keys and constraints, and avoiding unnecessary duplication help prevent anomalies and improve the accuracy and maintainability of the database.